Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: Step 3's `reason_node` quotes one excerpt word for word — good enough to prove the plumbing works, but a real answer often needs several excerpts combined and rephrased, which means a model has to write it. The risk the moment a model is writing is that it quietly fills gaps from its own training data instead of saying what the documents don't cover. So the upgraded Reasoner's prompt constrains it to use **only** the retrieved excerpts, cite each fact's source file, and use an exact fixed sentence when something isn't covered. It stays a separate node from the Retriever, so search quality and answer-writing quality can be checked and debugged apart. **If not:** when an answer is wrong, you couldn't tell whether the search found the wrong thing or the writer used the right thing badly.

After this step document questions get real, cited answers. Only `agents/reasoner_agent.py` and a new prompt file change — the graph, the state and every other node stay the same.

Solution — a strict, citing Reasoner with its own prompt

Story — `prompts/reasoner.txt`: this prompt carries three rules a schema alone couldn't enforce: answer only from the excerpts; name each fact's source file in brackets; and use an **exact**, fixed sentence when the excerpts don't cover the question. **If not:** an unprompted model tends to fill the gap from its own memory instead of admitting the excerpts fall short.

`src/prompts/reasoner.txt`

```
You answer questions using ONLY the document excerpts below. After each
fact, name its source file in square brackets, like [refund_policy.md].
If the excerpts do not answer the question, reply exactly:
The documents don't cover this.

Excerpts:
{sources}

Question: {task}
```

Changed from Step 3: `reason_node` keeps its name, its input (the state) and its output (`answer` + its path entry); inside, a Doc05 chain now writes the answer. `format_sources()` is new.

Story — `agents/reasoner_agent.py`: `format_sources()` puts each excerpt's file name directly in front of its text before it ever reaches the prompt, so the model *can* cite it — it has no other way to know which file a fact came from. The node only writes the answer — it doesn't search (that's the Retriever's job) and doesn't approve (that's Step 5's). **If not:** the model would have the excerpt text but no file names to cite, making the citation rule impossible to follow.

```
# src/agents/reasoner_agent.py
# The Reasoner agent: reason_node() answers only from the found excerpts
# and names the file each fact came from.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.models.llm import get_model
from src.utils.prompts import load_prompt

# new in Step 4
def format_sources(sources):
    text = ""
    for source in sources:
        text = text + f"[{source['source_file']}] {source['text']}\n\n"
    return text

# changed in Step 4: same name and output, now written by the model
def reason_node(state):
    prompt = ChatPromptTemplate.from_template(load_prompt("reasoner"))
    chain = prompt | get_model() | StrOutputParser()
    answer = chain.invoke({
        "sources": format_sources(state["sources"]),
        "task": state["task"],
    })
    return {"answer": answer, "path": ["reason_node"]}
```

Changed from Step 3: the integration test adds case 7 — the answer must name the file it came from.

Story — `tests/integration/test_live_graph.py`: citing the source file is this step's core promise, so it gets its own check against the real model: the refund answer must literally contain `[refund_policy.md]`. **If not:** a prompt change that quietly stopped the citations would go unnoticed — and in Step 5, every answer would then fail the citation check and need a human.

```
# tests/integration/test_live_graph.py
# Runs the whole graph against the real OpenAI API — each step adds cases.
```

```
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_graph
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from src.config import load_config
from src.graph import build_graph
from src.state import start_state

ACTION_TASK = "What is the refund policy? Send it to the customer."


def run(graph, thread_id, task, decision="yes"):
    config = {"configurable": {"thread_id": thread_id}}
    outcome = graph.invoke(start_state(task), config)
    if decision is not None and "__interrupt__" in outcome:
        outcome = graph.invoke(Command(resume=decision), config)
    return outcome


def show(case, passed, outcome):
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}: {outcome.get('path')}")


def test_general_question(graph):
    outcome = run(graph, "t1", "What is the capital of France?")
    passed = "Paris" in outcome["answer"] and "answer_node" in outcome["path"]
    show("case 1 (general question)", passed, outcome)


def test_action_pauses(graph):
    outcome = run(graph, "t2", ACTION_TASK, decision=None)
    show("case 2 (pauses)          ", "__interrupt__" in outcome, outcome)


def test_human_says_yes(graph):
    outcome = run(graph, "t3", ACTION_TASK, "yes")
    passed = outcome["approved"] and "human_approved" in outcome["path"]
    show("case 3 (human: yes)      ", passed, outcome)


def test_human_says_no(graph):
    outcome = run(graph, "t4", ACTION_TASK, "no")
    passed = outcome["answer"].startswith("Not approved")
```

```
    show("case 4 (human: no)       ", passed, outcome)


def test_grounded_answer(graph):
    outcome = run(graph, "t5", "What is the refund policy?")
    passed = outcome["grounded"] and "retrieve_node" in outcome["path"]
    show("case 5 (grounded)        ", passed, outcome)


def test_not_in_documents(graph):
    outcome = run(graph, "t6", "What is the policy on remote work?")
    passed = outcome["path"][-1] == "no_grounding_node"
    show("case 6 (not found)       ", passed, outcome)


# new in Step 4
def test_answer_cites_its_file(graph):
    outcome = run(graph, "t7", "What is the refund policy?")
    passed = "[refund_policy.md]" in outcome["answer"]
    show("case 7 (cites its file)  ", passed, outcome)


if __name__ == "__main__":
    load_config()
    graph = build_graph(MemorySaver())
    test_general_question(graph)
    test_action_pauses(graph)
    test_human_says_yes(graph)
    test_human_says_no(graph)
    test_grounded_answer(graph)
    test_not_in_documents(graph)
    # new in Step 4
    test_answer_cites_its_file(graph)
```

Expected output — cases 1-6 as in Step 3, then:

```
case 7 (cites its file)   -> OK: ['clean_task', 'retrieve_node',
'reason_node', 'human_approved']
```

Run it: the tests above, then `python main.py`. Check a few answers **by hand** against the documents too; "it ran" doesn't prove "it's grounded".

```
You: How long do refunds take to arrive?
  -- needs your approval --
  answer: Refunds are paid back to the original payment method within 5
```

```
working days after the returned product arrives [refund_policy.md].
  why:   every answer is checked by a person for now
  Approve? (yes/no): yes
  path: clean_task -> retrieve_node -> reason_node -> human_approved
```

(The answer is wrapped here to fit the page.)